

In the second article in the series, we start with a recap of the reasons that led to the definition and development of the second generation of Web protocols. Then, we dig further into SPDY, HTTP 2 and TLS 1.2.

The Web is still growing and evolving. So, in the third and last article in the series, we turn our attention to the proposed or recently standardised newer protocols including DNS over HTTPS, TLS 1.3, QUIC, and HTTP 3.

Terminology: Protocol stack

The network infrastructure (proxies, CDNs, firewalls or load balancers) and network applications (browsers or Web servers) have one thing in common — the protocol stack. Since this stack is such a ubiquitous entity, we will spend some time trying to describe it with some precision.

The first step is to understand why a ‘protocol stack’ is needed. A ‘protocol’ is required when components developed independently — either by different teams, organisations, countries or a mix of these — need to interoperate and deliver the same experience as if they were developed together. In our context, the word ‘protocol’ is only applied to ‘network protocol’, and the components referred to are those that provide communication features. For example, a

browser allows one to access an online encyclopaedia, a Web mail client allows one to communicate with other humans, and so on. In these cases, typically, the browser and the online encyclopaedia are not developed by the same organisation, as the expertise required to build these services is vastly different. However, users accessing the same encyclopaedia from different browsers must see the same content and get the same look-and-feel. Moreover, users may access the encyclopaedia from different networks (work/home/coffee shop/car) or different devices (desktop/handheld). In all these cases, not only must the user be able to access the content seamlessly, but also have a consistent experience. ‘Protocols’ play a big part in making this happen.

The other word in the term ‘protocol stack’ is ‘stack’. The early designers of networking protocols foresaw the complexities of network communication and proposed a layered approach to tame this complexity. In this approach, each layer addresses a set of requirements for communication, and provides those as services to the next higher layer. In this manner, a ‘stack’ of layers provides the full set of functionalities. Now, juxtaposing these two words, gives a layered solution, where each layer implements one or more protocols.

The big picture

Now, we turn our attention to the end-to-end flow of information from the client to the server and back, as depicted in Figure 1. In this diagram, the browsers on the left are attempting to retrieve content from the Web server on the right. When the client system is started afresh, the browser may not even be aware of the network address of the Web server. So, it all starts with the browser converting the hostname to a network address, which is accomplished with the DNS protocol. Once this conversion is done, the actual process of Web access is delivered by three key protocols: IP, TCP and HTTP. IP, the lowest of these three layers, handles addressing and routing of packets from the source to the destination, while TCP adds reliability on top of the functionality provided by IP. Finally, the HTTP protocol is the one that allows a browser to request for the required content and receive the response sent by the Web server. All the different types of Web content like HTML, JavaScript, CSS and a variety of other content are all transported using HTTP.

Figure 1 shows the different steps in communication, and it is worth reviewing these closely.

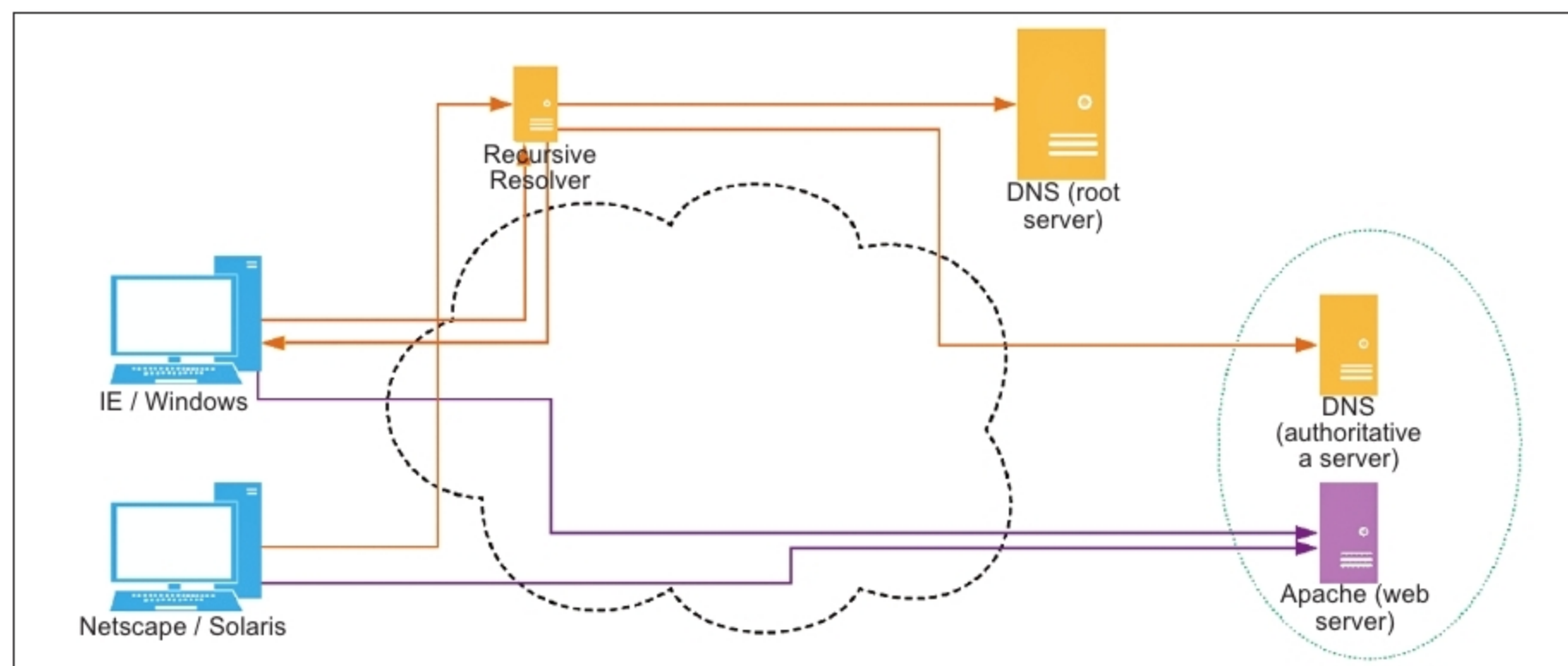


Figure 1: End-to-end Web request